PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA
Ministry of Higher Education and Scientific Research
University Center El Cherif Bouchoucha - Aflou
Institute of Sciences
Computer Science Department

Dr. SELLAM Abdellah

Lab 01: Preprocessing & Text Manipulation

String Operations, File I/O, and Regular Expressions in Python

1 Learning Objectives

By the end of this lab session, students should be able to:

1. Handle text encoding, multi-lingual scripts, and file I/O safely in Python.
2. Implement basic text normalization, sanitization, and tokenization without relying on external NLP libraries.
3. Construct complex Regular Expressions (`regex`) to parse, filter, and structure raw, unstructured text datasets.
4. Design an end-to-end preprocessing pipeline for downstream NLP tasks.

2 Part 1: String Operations & Text Encoding

2.1 1.1 Robust File I/O and Encodings

In real-world NLP applications, text datasets are gathered from heterogeneous sources. Always specify encoding explicitly when reading or writing files to prevent cross-platform issues.

```
# Reading and writing UTF-8 encoded files safely
input_path = "dataset_raw.txt"
output_path = "dataset_clean.txt"

with open(input_path, "r", encoding="utf-8") as f:
    raw_text = f.read()

# Basic string normalization
lowercased_text = raw_text.lower()
stripped_lines = [line.strip() for line in raw_text.splitlines() if line.
    strip()]
```

2.2 1.2 Exercise 1: Basic String Manipulation

Task: Given a raw text string with irregular spacing and mixed cases:

1. Write a function to collapse all redundant spaces and newline characters into a single space.
2. Split the text into naive sentences using delimiter heuristics (`.`, `!`, `?`).
3. Compute the vocabulary size (number of unique word tokens) using Python's `set` and basic whitespace splitting.

3 Part 2: Regular Expressions (`re` Module)

Regular expressions provide a deterministic mechanism for pattern matching, text extraction, and sanitization before feeding text into embedding models or neural networks.

Pattern	Description
`\w` / `\W`	Word character (alphanumeric + _) / Non-word character
`\d` / `\D`	Digit / Non-digit
`\s` / `\S`	Whitespace character / Non-whitespace character
`[a-zA-Z]`	Character class (matches any single specified character)
`+` / `*` / `?`	Quantifiers: 1 or more / 0 or more / Optional (0 or 1)
`(?P<name>...)`	Named capturing group

Table 1: Essential Regular Expression Syntax

3.1 2.1 Regex Metacharacters Summary

3.2 2.2 Practical Pattern Extraction

Below is a demonstration using Python's built-in `re` library to isolate specific domain patterns:

```
import re

text = """
Contact us at support@nlp-lab.edu or admin_dept@university.org.
Order #12344 processed on 2026-10-20. Visit https://nlp.example.com/dataset
    ?id=99.
"""

# Extracting Emails
emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', text
    )
print("Emails:", emails)

# Extracting URLs
urls = re.findall(r'https?://[^\s]+', text)
print("URLs:", urls)

# Masking ISO Dates
masked_text = re.sub(r'\d{4}-\d{2}-\d{2}', '[DATE]', text)
print("Masked:", masked_text)
```

4 Part 3: Practical Exercises & Challenges

4.1 Exercise 2: Web Scrape Sanitization Pipeline

Implement a function `preprocess_web_text(text: str) -> str` that sequentially executes the following transformation pipeline:

1. Strips out HTML tags (e.g., `<p>`, `<a href="...">`, `<br/>`).
2. Replaces all web URLs with the special token `<URL>`.
3. Replaces all email addresses with the special token `<EMAIL>`.
4. Replaces sequences of numeric digits with the token `<NUM>`.
5. Removes punctuation marks while preserving internal word boundaries and spaces.
6. Returns the fully lowercased, normalized string.

4.2 Exercise 3: Regex-Based Tokenizer

Standard whitespace splitting fails to isolate punctuation attached to words (e.g., `"Hello, world!"` → `["Hello,", "world!"]`).

Task: Construct a customized regex-based tokenizer `regex_tokenize(text: str) -> list[str]` using `re.findall()` or `re.finditer()` that handles:

- English contractions (e.g., `"don't"` → `["do", "n't"]` or `["don't"]`).
- Words containing internal hyphens (e.g., `"state-of-the-art"`).
- Standalone punctuation marks kept as distinct tokens.

4.3 Exercise 4: Log File Parsing using Named Groups

Assume you are provided with a dataset file `chat_logs.txt` formatted as follows:

```
[2026-10-20 14:02:11] User123: Hello! Has anyone tried the transformer
   model?
[2026-10-20 14:03:05] Bot_Assistant: Yes, see documentation at https://docs
   .ai/models.
[2026-10-20 14:05:22] Researcher_M: Check paper arXiv:2401.12345v1 for
   benchmarks!
```

Task:

1. Read and parse the file line by line.
2. Use named regular expression capturing groups (`(?P<timestamp>...)`, `(?P<username>...)`, `(?P<message>...)`) to convert each entry into a structured dictionary.
3. Extract all mentioned arXiv identifiers and external URLs into a separate summary report file named `extracted_links.txt`.

5 Discussion Questions

1. **Multilingual Preprocessing:** How do standard ASCII regular expressions (`\w+`) behave when applied to non-Latin scripts (e.g., Arabic, Cyrillic, Chinese)? How does Unicode normalization (NFC/NFD) impact text processing?
2. **Deterministic vs. Learned Tokenization:** What are the trade-offs of using rule-based/regex tokenizers versus statistical subword algorithms like Byte-Pair Encoding (BPE) or WordPiece in modern Language Models?